From Traces to Guarded Programs: Evidence-Gated Compilation of Recurrent Agent Workflows

Anonymous authors
Paper under double-blind review

Abstract

Tool-using agents often revisit the same read-only evidence path while paying for a model decision at every tool boundary. Replacing a recurring model-mediated prefix is tempting, but not ordinary caching. A later tool argument may encode a model decision that cannot be recovered from observed state; an apparently read-only call may cross an effect boundary; and a program that replays tool calls correctly may still change the answer produced by the remaining agent. The central question is therefore not whether a trace repeats, but whether the available evidence justifies substituting the model.

We introduce guarded agentic compaction (GAC), a trace-to-program compiler for this decision. GAC normalizes recorded episodes, mines recurrent read-only prefixes, reconstructs every tool argument from entry state or prior results, and synthesizes a bounded deterministic program with manifest, input, and output guards. It deploys a candidate only when a finite-sample selective-risk test certifies its error bound; otherwise it retires the candidate and runs the unchanged agent. On three live GitHub workflow families, GAC preserves 90/90 exact held-out contracts while reducing provider requests by 66.6% and tokens by 63.1% in aggregate. On a time-forward five-repository extension it completes four repositories and retires one; on NESTFUL and API-Bank it retires every recurrent family despite successful provenance, synthesis, and replay. Thus recurrence identifies a candidate, whereas evidence determines whether replacing the model is justified.

1 Introduction

The standard agent loop pays for a model decision at every tool boundary: model, tool, model, tool, and so on (Yao et al., 2023). That flexibility is valuable while a workflow is novel. At scale, however, mature agents often revisit the same read-only evidence path with the same data dependencies. In those settings, repeating the model decision adds latency, tokens, and cost without necessarily adding useful adaptation.

Removing that decision is not ordinary caching. A tool argument may depend on a specific earlier observation, a repeated sequence may cross an approval or write, an apparently read-only tool may hide an undeclared effect, and a program that replays the same tool outputs may still change the downstream answer. The problem is therefore not just "find recurring traces." The problem is to decide when a recurrent model-mediated region can be replaced by a deterministic program and when it should be refused.

Consider held-out issue #4420 in the expanded 30-record study. Given only `issue_number=4420`, the observed trace reads `record(4420)`, `labels(4420)`, and `comments(4420, limit=100)` before rendering an auditable triage answer. A recurrence-only optimizer would emit all three reads. GAC instead retires that candidate because `comments.limit` varies across traces and has no consistent provenance expression; it emits only the grounded `record -> labels` prefix and returns control to the agent. That two-read artifact passes 92/92 calibration groups at $\alpha = .05$ (upper bound 0.0498).

Why stop there? A separate, earlier 18-record study supplies the counterexample: its aggressive three-read artifact replayed all 45 tool calls, yet issue #6602 lost the URL from a Markdown link and only 17/18 downstream answers remained exact. A provider-free continuation guard detects and checked-renders that miss; it is a mechanism check, not live safety evidence. Together, the two cohorts show

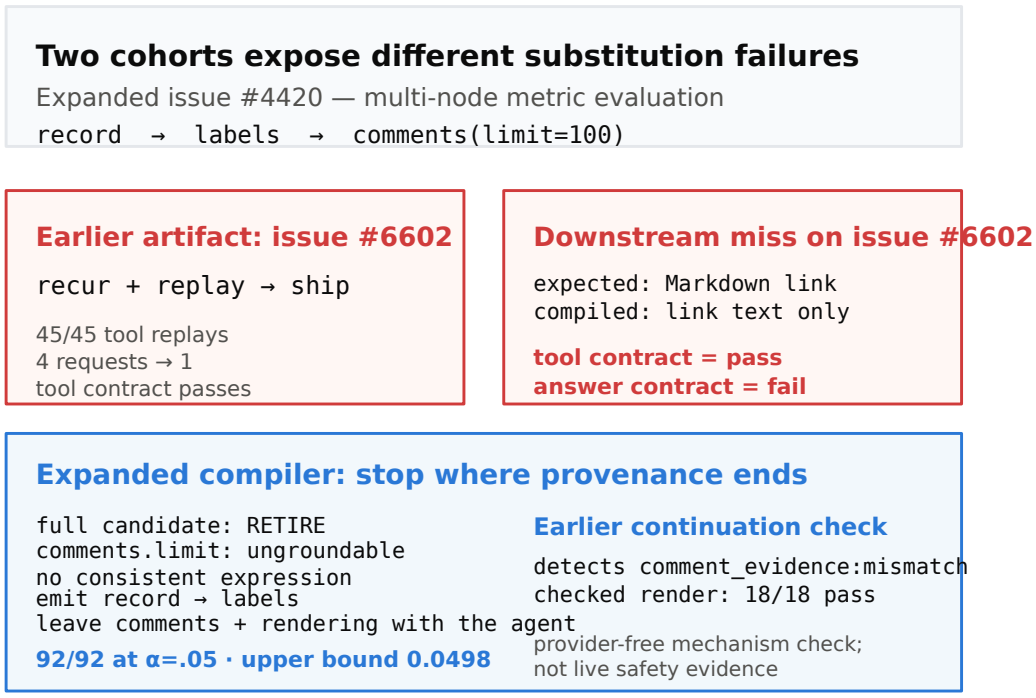

Figure 1: Two retained cohorts expose different failure modes. In the expanded study, issue #4420 forces a shorter grounded prefix; in the earlier study, issue #6602 shows that clean tool replay need not preserve the downstream answer.

why recurrence proposes a candidate, while provenance, continuation scope, and statistical evidence determine what may replace the model.

Framed this way, the closest precedent is not prompt optimization but the tracing just-in-time compiler. Dynamo (Bala et al., 2000) detects a hot path, compiles it behind a *guard*, and deoptimizes to the interpreter when the guard fails (Hölzle et al., 1992). Agent traces admit the same structure — hot-trace detection becomes family mining, guard synthesis becomes an entry contract, and deoptimization becomes fallback to the unchanged agent — but they add three obligations a JIT does not face. Tool arguments must be shown to be *grounded* in observable state rather than invented by the model; declared *effects* must permit speculation, because an agent's "interpreter" can call the outside world; and the residual error rate of a compiled region is statistical, so it needs a finite-sample bound rather than a soundness proof.

We study these obligations through *guarded agentic compaction* (GAC), a compile-or-retire trace-to-program system for recurrent read-only prefixes. The method normalizes executions into a typed episode representation, reconstructs value provenance for tool arguments, mines recurrent prefix families, synthesizes bounded programs, induces runtime guards and verifiers, and admits an artifact only when a finite-sample selective-risk calculation supports it. Otherwise the baseline agent is retained unchanged (fig. 2). The core contribution is thus not "agent compilation" in general, but an admissibility argument for trace-derived specialization.

The evidence is deliberately multi-sided. On three distinct GitHub workflow families with live provider calls, compiled programs preserve or improve exact held-out task outcomes while reducing requests, tokens, latency, and cost; a narrower time-forward task then tests transfer across repositories and retains both success and compile-time retirement. NESTFUL and API-Bank show the opposite regime, in which recurrent executable traces still fail to justify deployment. We also retain the strongest practical counterpoint: when a workflow is already known, hand-written programs can be competitive or tied, so any claim of automatic superiority would be incorrect.

The paper makes three contributions:

1. A guarded specialization pipeline (Alg. 1) that combines typed provenance, effect and position barriers, bounded synthesis, runtime verification, and finite-sample compile-or-retire admission (Alg. 2, proposition 1).
2. A real-provider evaluation across three public-record workflow families, plus a narrower cross-repository, time-forward extension that keeps both successful artifacts and principled retirements.
3. A negative-result boundary showing that recurrence and replayability do not themselves license deployment: NESTFUL and API-Bank retire under the same admission logic, and manual programs constrain any runtime-dominance claim.

2 Problem Formulation

Episodes and candidate regions. An episode $E = (z, M, e_{1:T}, y)$ contains an entry-state snapshot z, a content-addressed execution manifest M, ordered events e_t (model requests and responses, tool calls and results, handoffs, guardrails, approvals, errors, commit boundaries), and an observable outcome y. Each tool f carries an application-owned effect declaration $\epsilon(f)$ and capability flags such as *speculatable*; an unknown effect is not a read.

A candidate region $R = [a, b]$ begins at a model-request boundary and ends after a tool result. It is *groundable* when every call argument is an expression over entry state or prior results inside R,

$$\forall c_j \in R,\ \forall u \in \mathrm{args}(c_j) : \quad u = g_{j,u}\big(z, o_{<j}\big), \qquad g_{j,u} \in \mathcal{L}, \tag{1}$$

for a closed, bounded transform library $\mathcal{L}$; and *effect-admissible* when every tool in R is declared read-only, speculatable, replayable, and inside one principal and isolation partition. Handoffs, approvals, writes, errors, and unknown effects are hard barriers, not costs to be traded away. Because the deployed runtime resolves artifacts at the initial model boundary, the compiler also imposes a *position invariant*:

$$a = 0 \quad \text{in the normalized model-boundary index.} \tag{2}$$

This is not aesthetic. A suffix program dispatched at entry can reorder or duplicate calls the agent has already made — a semantic mismatch between compilation-time and resolution-time position rather than a tuning failure, and a pilot that violated eq. (2) produced exactly that fault (appendix E).

Selective optimization objective. The compiler emits an artifact $A = (P, H, V, q, \eta, M)$: a deterministic program P, hard guard H, output verifier V, nonconformity score q, threshold η, and manifest pins M. On a future episode with entry state z, dispatch is *selective*,

$$d_A(z) = \mathbf{1}\{\, H(z) = 1 \ \wedge\ q(z) \le \eta \ \wedge\ M \text{ matches} \,\}, \tag{3}$$

and any failed precondition runs the unchanged baseline agent B. If execution fails before a commit and staging proves the attempt clean, B runs; a dirty failure after a commit is an incident, not a fallback. Writing $L(A, z) \in \{0, 1\}$ for a wrong compiled execution *after* dispatch and $C(\cdot, z)$ for execution cost, we seek

$$\max_{A}\ \mathbb{E}\big[d_A(z)\,\{C(B, z) - C(A, z)\}\big] \quad \text{s.t.} \quad \Pr[L(A, z) = 1 \mid d_A(z) = 1] \le \alpha. \tag{4}$$

Two properties shape everything that follows. The constraint conditions on dispatch, so abstention is free and only post-dispatch errors are charged — the reject-option formulation applied to a compiler rather than a classifier — and returning *no* artifact is feasible, so RETIRE is a legitimate output rather than a failure mode. Equation (4) is a design target: the implementation returns the first family that survives every stage of Alg. 1, and we prove no bound on the gap to the best one. Calibration uses independent scenario groups as its statistical units: a group is admitted if any member dispatches and violates if any dispatched member is wrong. This conservative group risk equals episode risk in the reported live studies, where each group is one public record.

3 Guarded Agentic Compaction

Figure 2 and Alg. 1 give the same six stages at two levels of detail. We describe each and state what it can refuse.

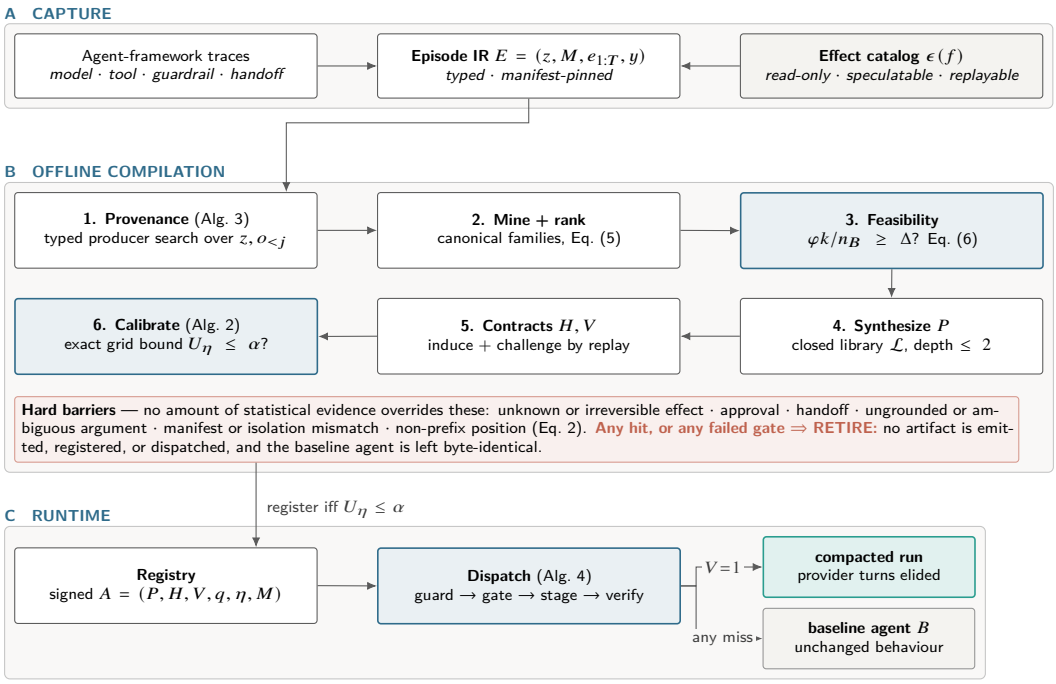

Figure 2: Architecture of GAC. **(A)** Capture normalizes traces into one typed Episode IR against a declared effect catalog. **(B)** Offline compilation is a cascade of independent rejection points; the shaded band lists barriers no statistical evidence can override. **(C)** At runtime exactly one terminal edge compacts; the rest return the unchanged agent.

1. Typed provenance. A capture adapter normalizes framework traces into the Episode IR; manifests pin prompt, policy, guardrail, tool, model, SDK, tracer, entry-contract, and effect-catalog identities, and episodes with different compatibility keys are partitioned before any learning. For each call-argument slot, the program-argument trace graph (PATG, Alg. 3) searches entry-state and prior-result paths for typed values reachable by a bounded transform — execution provenance in the database sense (Cheney et al., 2009), specialized to tool arguments. Two outcomes block a region rather than guessing: a slot with *no* witness is a genuine model decision, and a slot with more than κ witnesses is ambiguous.

2. Region mining and ranking. The miner enumerates model-boundary-to-result windows of at most $w_{\max}$ tool calls, qualifies effects and live-ins, and hashes tool signatures, dependency topology, run shape, and live-in/live-out shape while abstracting literal values. Families require support across independent scenario *groups* rather than raw event frequency, which prevents a single chatty scenario from manufacturing recurrence. Survivors are ranked by a minimum-description-length trade-off that pays for expected savings and charges for heterogeneity, effect exposure, and program size:

$$\text{score}(F) = \underbrace{s_F\, \bar{k}_F\, c_m}_{\text{expected saving}} - \lambda_1 H(F) - \lambda_2 \rho_{\text{eff}}(F) - \lambda_3 |F|, \tag{5}$$

with s_F the group support, $\bar{k}_F$ the mean removable model requests, c_m their unit cost, and $H(F)$ the Shannon entropy over canonical variants inside the family; without λ_1 the ranking prefers a heterogeneous family whose single canonical form fits none of its members. Equation (5) affects only *which* family is tried first — no admission decision depends on it.

3. Feasibility ceiling. Before any synthesis runs, an estimator bounds what the compiler could achieve with a perfect verifier and a gate that never abstains:

$$\Delta_{\max} = \frac{\varphi\, k}{n_B}, \qquad \text{necessary:} \quad \varphi\, \rho\, k \;\geq\; \Delta\, n_B, \tag{6}$$

Algorithm 1 COMPILE — the compile-or-retire cascade. Every exit is either a registered artifact or an attributed RETIRE, so rejections stay in the denominator. Stages marked (CALLER) are separate library entry points run in this order rather than steps inside one function.

Input: episodes $\mathcal{E}$; effect catalog C; entry schema Θ; budgets α, δ; grid Λ; feasibility target Δ
Output: artifact $A = (P, H, V, q, \eta, M)$, or RETIRE with an attributed reason

$\mathcal{E} \leftarrow \textsc{Qualify}(\mathcal{E}, C)$; $\{\mathcal{E}_M\} \leftarrow \textsc{PartitionBy}(\mathcal{E}, \text{manifest, principal, isolation})$ ▷ drop partial runs, unpinned manifests
for all partitions $\mathcal{E}_M$ **do**
  $(\mathcal{T}, \mathcal{D}, \mathcal{K}, \mathcal{S}) \leftarrow \textsc{GroupedSplit}(\mathcal{E}_M)$ (CALLER) ▷ train / dev / calibration / sealed test, split by *group*
  $G \leftarrow \textsc{BuildPatg}(\mathcal{T}, \Theta)$ ▷ Alg. 3: block ungrounded or ambiguous slots
  $\mathcal{F} \leftarrow \textsc{Canonicalize}(\textsc{MineRegions}(G, C, w_{\max}))$, keeping only cross-group support
  if $\varphi k / n_B < \Delta$ **then** (CALLER)
    record RETIRE **(infeasible ceiling, Eq. 6)**; **continue** ▷ try the next compatible partition
  end if
  for all families $F \in \mathcal{F}$ ranked by Eq. (5) **do**
    $P \leftarrow \textsc{Synthesize}(F, \mathcal{L})$ at depth ≤ 2; **if** $P = \perp$ **then continue** ▷ ungroundable slot
    $(H, V) \leftarrow \textsc{InduceContract}(F)$; **if** $\neg\textsc{ReplayAndChallenge}(P, V, \mathcal{D})$ **then continue** ▷ counterexample
    $q \leftarrow \textsc{FitScore}(\mathcal{D})$; **freeze** q ▷ dev groups only; never calibration
    $\eta \leftarrow \textsc{Calibrate}(q, H, \mathcal{K}, \Lambda, \alpha, \delta)$; **if** $\eta = \perp$ **then continue** ▷ Alg. 2
    return $\textsc{Package}(P, H, V, q, \eta, M)$ with evidence and lifecycle
  end for
end for
return RETIRE

with n_B baseline model requests per episode, φ the fraction of episodes holding an eligible region, k the requests removed per successful dispatch, and ρ the verifier pass rate. Since eq. (6) assumes $\rho = 1$, no measured reduction can exceed it, and it is met with equality exactly when nothing abstains and nothing fails. Reporting it first makes a decisive negative answer cheap: a workload whose ceiling sits under the target can be declined without building a compiler for it.

4–5. Bounded synthesis and contracts. Synthesis searches a 23-operator library at depth at most two, covering path selection, indexing, string normalization, arithmetic, comparisons, bounded collection operations, and narrow loops; bindings are ranked by stability across groups, and a group-wise refit rejects bindings that exploit row-level coincidence. The hypothesis space is deliberately small enough to read: the approach resembles programming-by-example (Gulwani, 2011) and bounded sketching (Solar-Lezama et al., 2006), and it does *not* generate arbitrary Python. Hard guards then constrain entry types, required fields, categorical sets, numeric intervals, patterns, isolation keys, allowed effects, and manifest pins; verifiers constrain live-out presence, type, cardinality, empirical hulls, provenance, effect multisets, and call counts. These are candidate invariants, not proofs for all future inputs — which is why a statistical gate follows them rather than replacing them.

6. Exact risk-gated admission. Algorithm 2 decides deployment. The score q is fitted on *dev* groups only — never on calibration groups — from entry-observable features such as missing fields, hull margin, temporal drift, and provenance ambiguity, then frozen together with a finite threshold grid Λ. For each $\eta \in \Lambda$ the compiler counts admitted calibration groups (n_η) and those containing at least one post-dispatch violation (k_η), and inverts the one-sided exact binomial test (Clopper & Pearson, 1934) at a Bonferroni-corrected level, with explicit empty and all-violation cases:

$$U_\eta = \begin{cases} 1, & n_\eta = 0 \text{ or } k_\eta = n_\eta, \\ \mathrm{Beta}^{-1}\left(1 - \frac{\delta}{|\Lambda|}; k_\eta + 1, n_\eta - k_\eta\right), & 0 \le k_\eta < n_\eta, \end{cases} \qquad U_\eta\big|_{k_\eta = 0} = 1 - \left(\frac{\delta}{|\Lambda|}\right)^{1/n_\eta}. \quad (7)$$

It admits the largest-coverage threshold with $U_\eta \le \alpha$ and retires the family otherwise — a fixed-grid instance of the learn-then-test discipline (Angelopoulos et al., 2022). The closed form on the right is the operative constraint in our experiments: at $\alpha = 0.05$, $\delta = 0.1$, and $|\Lambda| = 11$, a zero-violation family still needs $n_\eta \ge 92$ admitted groups. The default output is refusal, not emission.

Proposition 1 (Per-candidate selective-risk admission)**.** *Fix one candidate program, hard guard, verifier, score, group-level violation rule, and finite grid Λ before calibration. If calibration groups are i.i.d. from the future-group distribution, then with probability at least $1 - \delta$, every $\eta \in \Lambda$ satisfies*

Algorithm 2 CALIBRATE — fixed-grid exact selective admission. The score q and the grid Λ are frozen *before* this procedure sees a calibration group, which is what makes the union bound legitimate.

Input: frozen score q; hard guard H; calibration groups $\mathcal{K}$; grid Λ; budgets α, δ
Output: threshold η with a simultaneous guarantee, or $\perp$ (RETIRE)

$\mathcal{A} \leftarrow \emptyset$ ▷ admissible (coverage, threshold) pairs
for all $\eta \in \Lambda$ **do**
  $K_\eta \leftarrow \{ g \in \mathcal{K} : \exists z \in g,\ q(z) \le \eta \ \wedge\ H(z) = 1 \}$; $n_\eta \leftarrow |K_\eta|$; **if** $n_\eta = 0$ **then continue** ▷ vacuous bound $U_\eta = 1$
  $k_\eta \leftarrow |\{ g \in K_\eta : g \text{ contains a violation} \}|$ ▷ *wrong after dispatch* only, never abstention
  $U_\eta \leftarrow 1$ if $k_\eta = n_\eta$, else $\mathrm{Beta}^{-1}\big(1 - \frac{\delta}{|\Lambda|};\ k_\eta + 1,\ n_\eta - k_\eta\big)$; **if** $U_\eta \le \alpha$ **then** $\mathcal{A} \leftarrow \mathcal{A} \cup \{(n_\eta/|\mathcal{K}|, \eta)\}$ ▷ Eq. (7)
end for
return $\perp$ if $\mathcal{A} = \emptyset$, else $\arg\max_{(\mathrm{cov},\eta)\in\mathcal{A}}\ \mathrm{cov}$ ▷ largest coverage among certified thresholds

$r_\eta \le U_\eta$, where r_η is post-dispatch group risk. Hence the selected threshold satisfies $r_\eta \le \alpha$ whenever $U_\eta \le \alpha$.

Appendix A gives the proof. The guarantee is deliberately narrow: it is *per fixed candidate*, not compiler-wide over the adaptive candidate search Alg. 1 actually performs, and it conditions on group exchangeability rather than establishing it (section 7). The scientific point is that guarded deployment requires an explicit evidence boundary, not just a recurrent trace and a successful replay.

4 Experimental Setup

Because the claim is an *admissibility* claim, the evaluation asks three questions: does specialization preserve outcomes while saving work on real records (§5.1)? Does it transfer under a time-forward split (§5.2)? And does it refuse when evidence is thin (§5.3)?

4.1 Benchmark at a glance

Every GitHub sample begins with one issue or pull-request number from a pinned public snapshot. The agent must make read-only calls and return exact, source-grounded fields, not a subjective preference. The three primary families use 132 discovery and 30 balanced held-out records each, with live provider calls. Table 1 shows what one sample asks an agent to read and return; it tests both the structured answer and whether its evidence remains available after compaction.

Table 1. Primary GitHub benchmark at a glance. Each sample supplies one record number from a pinned public snapshot; every listed answer field is checked against the source record.

Task	Read-only evidence path	Exact answer contract
Issue type	`issue_number` → record, official labels, comments	Title, state, label-derived category, verbatim excerpt
PR outcome	`pr_number` → PR record, merge status, discussion	`open`, `merged`, or `closed_unmerged`, plus title and comment excerpt
Backlog attention	`issue_number` → open-issue record, assignees, discussion	`owned`, `discussed_unowned`, or `awaiting_first_response`, plus title, owner, and excerpt

Cross-repository time-forward extension. A narrower two-read PR-outcome task spans five frozen public repositories. Held-out records are strictly newer than discovery records, and selection is sealed before provider calls; repositories without an admitted artifact remain in the result. A balanced rerun on three repositories controls the `open`/`merged`/`closed_unmerged` mix.

External refusal benchmarks. NESTFUL and API-Bank test valid executable traces whose evidence is still insufficient. They retain the argument-level producer structure needed to measure provenance, synthesis, replay, and admission—not live planning quality (Basu et al., 2025; Li et al., 2023).

Table 2: Primary live-provider results ($n = 30$ held-out records per family). "Interfaces" counts provider-visible tool interfaces. All conditions see identical records, model, and grader; *Manual* is hand-written with full knowledge of the workflow.

	Exact held-out contract			Reduction of compiled vs. baseline (%)				
Family	Base	Compiled	Manual	Requests	Interfaces	Tokens	Latency	Cost
Issue-type routing	30/30	**30/30**	30/30	50.0	0.0	39.5	51.7	32.0
PR-outcome audit	30/30	**30/30**	30/30	75.0	66.7	80.8	73.0	75.3
Backlog-attention routing	29/30	**30/30**	30/30	74.8	66.3	81.4	68.9	75.1
Weighted total ($n = 90$)	89/90	**90/90**	90/90	66.6	44.2	63.1	64.2	58.7

Table 3: Cross-repository time-forward results on the narrower two-read pull-request outcome task. Held-out records are strictly newer than discovery records and selection is sealed before any provider call. The 5-repo cohort retires `pytorch/pytorch`; a fixed template is at least as efficient in both settings.

		Exact task contract			Reduction vs. baseline (%)			
Setting	Repos	Discovery	Held-out	Classes	Req.	Tokens	Latency	Cost
5-repo core	4/5	580/580	120/120	skewed	44.4	52.4	49.4	48.6
3-repo rerun (balanced)	3/3	360/360	180/180	balanced	66.7	78.4	60.7	72.7

Conditions, metrics, and statistics. Each GitHub family compares the unchanged baseline B, the compiled artifact, and a hand-written pre-model program on the same records. Quality is exact task-contract satisfaction; resources are requests, interfaces, tokens, latency, and estimated cost. We use paired exact McNemar tests, bootstrap intervals, and Wilcoxon signed-rank tests; the only guaranteed multiplicity control is the Bonferroni term in eq. (7). All admission runs use $\alpha = 0.05$, $\delta = 0.1$, and $|\Lambda| = 11$; appendix C lists the remaining settings.

5 Results

5.1 Primary workflows: preserved contracts, less work

Across the 90 held-out records, GAC satisfies 90/90 exact contracts versus 89/90 for the unchanged agent (table 2). It reduces provider requests by 66.6%, tokens by 63.1%, latency by 64.2%, and estimated cost by 58.7% in aggregate. The per-family pattern is intuitive: issue-type routing admits only a two-read prefix, whereas the two deeper workflows approach 75% fewer requests.

No held-out record is a compiled-only failure; the one-sided 95% upper bound on that discordance is 3.3%. This supports preservation on the observed cohort, not equivalence in general. Each artifact was admitted with 92 zero-violation calibration groups ($U_\eta = 0.0498 \leq .05$). Hand-written programs also achieve 90/90 and are cheaper for issue-type routing, so the evidence supports automatic discovery and guarded lifecycle management—not runtime dominance over correct manual code.

5.2 Time-forward transfer: compile or retire

In the five-repository cohort, GAC completes 120/120 exact held-out pairs on four repositories and retires `pytorch/pytorch` at compile time rather than loosening its gate (table 3). The balanced three-repository rerun reaches 180/180 exact pairs with 66.7% fewer requests. A fixed two-read template is essentially tied, so this is evidence of automatic discovery, validation, and principled retirement—not that learning dominates engineering.

5.3 Refusal when evidence is insufficient

The external benchmarks expose the opposite outcome. NESTFUL clears provenance, synthesis, and held-out replay: 12 families synthesize, with 24 replays passing and none wrong. Yet its largest

Table 4: Where refusal happens. Both benchmarks clear provenance, synthesis, and replay, then retire at calibration with *zero* wrong held-out executions: the binding constraint is sample size, not correctness. Stages refer to Alg. 1.

Stage of Alg. 1	NESTFUL	API-Bank
Episodes / recorded traces	1,415	212
Tools compilable / effect-blocked	—	21 / 28
Dependency slots with producer recovered (1)	5,531 / 5,746 (96.3%)	559 / 881 (63.5%)
Episodes yielding an admissible window (2)	1,207 (85.3%)	37 (17.5%)
Families reaching the support floor (2)	32	19
Families that synthesize a program (4)	12	2
Held-out replay: pass / abstain / **wrong** (5)	24 / 12 / **0**	0 / 2 / **0**
Largest family support $n_{\max}$ (6)	26	8
Groups required by Eq. (7) (6)	92	92
Artifacts admitted	**0**	**0**

family has only 26 groups, below the 92 zero-violation groups required by eq. (7), so every candidate retires (table 4). API-Bank fails earlier: its effect catalog blocks 28 of 49 tools before mining, leaving maximum support of eight; its two synthesized candidates also retire.

These outcomes separate trace-level properties from deployment evidence. Provenance reachability and replay correctness can identify a plausible program; they do not certify that it should replace a model. Refusal is the intended result when the sample cannot support the registered risk constraint, not an implementation failure.

6 Related Work

Guarded trace and agent workflow compilation. GAC adapts the tracing-JIT pattern—compile a hot path behind a guard and return to the interpreter on a miss (Bala et al., 2000; Hölzle et al., 1992)—to agents, whose arguments must remain grounded in observed state and whose speculation must be licensed by declared effects (Lucassen & Gifford, 1988). Recent systems mine meta-tools (AWO), induce workflows, cache plans, compile typed orchestration, validate tool-state plans, search workflow graphs, or prune multi-agent topology (Abuzakuk et al., 2026; Wang et al., 2025; Zhang et al., 2025; Wei et al., 2026; Winston et al., 2026; Li et al., 2026; Chen et al., 2026). They motivate workflow optimization; GAC's distinct claim is an evidence gate for whether recurrence licenses replacement, not unmeasured superiority.

Other optimization targets. DSPy optimizes prompts, GEPA evolves them, RouteLLM changes models, and LLMCompiler parallelizes current calls (Khattab et al., 2024; Agrawal et al., 2026; Ong et al., 2025; Kim et al., 2024). GAC instead replaces a historical boundary only after guarded evidence.

Synthesis, risk control, and evaluation. The bounded search draws on programming by example, sketching, and execution provenance (Gulwani, 2011; Solar-Lezama et al., 2006; Cheney et al., 2009); admission uses a Clopper–Pearson bound with fixed-grid learn-then-test, not a new bound (Clopper & Pearson, 1934; Angelopoulos et al., 2022). BFCL and ToolLLM score tool use broadly (Patil et al., 2025; Qin et al., 2023); NESTFUL and API-Bank retain the producer references and call results needed for post-trace provenance, replay, and refusal (Basu et al., 2025; Li et al., 2023).

7 Discussion and Limitations

The strongest supported claim is narrow: repeated read-only workflow prefixes can become guarded deterministic programs that preserve exact held-out contracts on real GitHub families while failing closed when evidence is inadequate, including on NESTFUL, API-Bank, and one cross-repository cohort.

Gate maturity is the main scientific risk. Proposition 1 is per-candidate, not compiler-wide over the adaptive search of Alg. 1; certification requires freezing the candidate relative to calibration. The gate is empirically step-like at $n_\eta \geq 92$: we show that it *refuses* correctly far better than that q *discriminates*. Refitting q for a genuine frontier is the key follow-up.

External validity and practical competitiveness. The richer evidence comes from one revision-pinned repository and one provider/model family; the cross-repository extension is a narrower, template-tied task. We establish no superiority on full cross-repository workflows, under drift, or across providers, and do not price manual authoring or maintenance against hand-written programs.

Semantic scope. Exact task contracts do not certify semantic equivalence once a model continuation follows; retained-continuation replay exposes that gap. Writes, handoffs, hosted tools, undeclared effects, and guardrail evaluations at a removed turn lie outside proposition 1: they are deployment boundaries, not implementation accidents.

8 Conclusion

GAC treats recurrence as candidate evidence, not permission to rewrite an agent. Across live GitHub families and a cross-repository extension, compile-or-retire saves resources while preserving exact held-out contracts; on NESTFUL and API-Bank it refuses. Judge workflow optimizers by their evidence, refusals, and fallback—not only the turns they remove.

AI Use Statement

In preparing this submission, the authors used generative AI tools to help restructure the manuscript, improve readability, suggest section organization, draft and revise parts of the paper text, critique methodological exposition and internal consistency, and convert repository-grounded results into LaTeX tables and submission materials. We did not use generative AI tools to generate experimental results or to replace manual verification of claims. All reported numbers, citations, equations, and comparisons were checked against the repository artifacts, retained outputs, and source files, and the authors take responsibility for the final content of the paper.

Ethics Statement

This paper studies methods for removing model-mediated decisions from tool-using agents. The main risk is over-trusting a compiled prefix and thereby accelerating unsafe automation or silently removing safeguards; deleting a model boundary can also delete a guardrail evaluation that happened to run at that boundary. Our method mitigates this risk by compiling only read-only prefixes, treating unknown effects, approvals, handoffs, and writes as hard barriers, and defaulting to retirement or fallback when evidence is insufficient. All study data are public records (GitHub repository metadata) or public benchmarks; no personal or private data were collected. The reported artifact is not a license to bypass human review, legal or compliance controls, or safety guardrails in high-stakes domains; unsupported regimes are part of the result, not exceptions to hide.

Reproducibility Statement

The main paper reports exact cohort sizes, evaluation protocols, quantitative metrics, and the claim boundaries attached to each study. Algorithms 1 and 2 give the deployed compiler and admission rule; Algorithms 3 and 4 in the appendix give provenance construction and runtime dispatch. Appendix A proves Proposition 1, and Appendix C lists every hyperparameter used by the reported runs. The repository contains source manifests, raw result files, deterministic table and figure generation, and scripts for the live GitHub studies, the cross-repository extension, NESTFUL, and API-Bank. For anonymous submission, the code and retained artifacts should be packaged as an anonymous supplementary archive with hashes and manifests preserved.

REFERENCES

Sami Abuzakuk, Anne-Marie Kermarrec, Rishi Sharma, Rasmus Moorits Veski, and Martijn de Vos. Optimizing agentic workflows using meta-tools. *arXiv preprint arXiv:2601.22037*, 2026. URL https://arxiv.org/abs/2601.22037.

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2507.19457. Oral presentation.

Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. Learn then test: Calibrating predictive algorithms to achieve risk control. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=TNcOrox3tQ.

Vasanth Bala, Evelyn Duesterwald, and Sanjeev Banerjia. Dynamo: A transparent dynamic optimization system. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 1–12, 2000. doi: 10.1145/349299.349303.

Kinjal Basu, Ibrahim Abdelaziz, Kiran Kate, Mayank Agarwal, Maxwell Crouse, Yara Rizk, Kelsey Bradford, Asim Munawar, Sadhana Kumaravel, Saurabh Goyal, Xin Wang, Luis A. Lastras, and Pavan Kapanipathi. NESTFUL: A benchmark for evaluating LLMs on nested sequences of API calls. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 33538–33547, 2025. doi: 10.18653/v1/2025.emnlp-main.1702. URL https://aclanthology.org/2025.emnlp-main.1702/.

Yulang Chen, Haoxuan Peng, Jinyan Liu, Zichen Wen, Dongrui Liu, and Linfeng Zhang. AgentSlimming: Towards efficient and cost-aware multi-agent systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pp. 30064–30086, 2026. doi: 10.18653/v1/2026.acl-long.1387. URL https://aclanthology.org/2026.acl-long.1387/.

James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009. doi: 10.1561/1900000006.

C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi: 10.1093/biomet/26.4.404.

Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pp. 317–330, 2011. doi: 10.1145/1926385.1926423.

Urs Hölzle, Craig Chambers, and David Ungar. Debugging optimized code with dynamic deoptimization. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 32–43, 1992. doi: 10.1145/143095.143114.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=sY5N0zY5Od.

Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An LLM compiler for parallel function calling. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 24370–24391, 2024. URL https://proceedings.mlr.press/v235/kim24y.html.

Junyan Li, Zhang-Wei Hong, Maohao Shen, Yang Zhang, and Chuang Gan. FlowCompile: An optimizing compiler for structured LLM workflows. *arXiv preprint arXiv:2605.13647*, 2026. URL https://arxiv.org/abs/2605.13647.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 3102–3116, 2023. doi: 10.18653/v1/2023.emnlp-main.187. URL https://aclanthology.org/2023.emnlp-main.187/.

John M. Lucassen and David K. Gifford. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pp. 47–57, 1988. doi: 10.1145/73560.73564.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/5503a7c69d48a2f86fc00b3dc09de686-Abstract-Conference.html.

Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 48371–48392, 2025. URL https://proceedings.mlr.press/v267/patil25a.html.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2023. URL https://arxiv.org/abs/2307.16789.

Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 404–415, 2006. doi: 10.1145/1168857.1168907.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 63897–63911, 2025. URL https://proceedings.mlr.press/v267/wang25bx.html.

Lei Wei, Qi Liu, Ruiyang Huang, Xiao Peng, Sicong Xie, Lanbo Lin, Chenhao Jiang, Yuanwu Xu, Tianyuan Yang, Jiayao Liu, Li Cai, Zhaolu Kang, and Bin Wang. EvoC2F: Compiling tool orchestration for efficient and evolvable LLM agents. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL https://openreview.net/forum?id=ZSGB91kMOG.

Caleb Winston, Ron Yifeng Wang, Azalia Mirhoseini, and Christos Kozyrakis. Agent JIT compilation for latency-optimizing web agent planning and scheduling. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL https://arxiv.org/abs/2605.21470.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

Qizheng Zhang, Michael Wornow, and Kunle Olukotun. Cost-efficient serving of LLM agents via test-time plan caching. *arXiv preprint arXiv:2506.14852*, 2025. URL https://arxiv.org/abs/2506.14852.

Algorithm 3 BUILDPATG — typed argument provenance. A slot with no witness is a genuine model decision; a slot with too many witnesses is ambiguous. Both block the region rather than defaulting to the cheapest explanation.

Input: episodes $\mathcal{T}$; entry schema Θ; transform library $\mathcal{L}$; ambiguity cap κ
Output: provenance graph G with one edge per grounded argument slot

$G \leftarrow \emptyset$
for all episodes $E = (z, M, e_{1:T}, y) \in \mathcal{T}$ **do**
  $\Sigma \leftarrow \{(\text{“}z\text{”}, \pi, v) : (\pi, v) \in \textsc{Flatten}(z),\ \pi \in \Theta\}$ ▷ only allowlisted entry paths are eligible
  for all tool calls $c_j \in e_{1:T}$ in order **do**
    for all argument slots $u \in \text{args}(c_j)$ **do**
      $\mathcal{H} \leftarrow \{\, \sigma \in \Sigma : \exists\, g \in \mathcal{L},\ g(\sigma) = u,\ \text{depth}(g) \le 2 \,\}$
      if u is declared *literal-only* for c_j **then**
        $\mathcal{H} \leftarrow \{\sigma \in \mathcal{H} : g = \text{id or } g \text{ constant}\}$ ▷ never reconstruct a page index or a limit
      end if
      if $\mathcal{H} = \emptyset$ **then**
        mark c_j MODEL-ORIGINATED; **block** region
      else if $|\mathcal{H}| > \kappa$ **then**
        mark c_j AMBIGUOUS; **block** region
      else
        $G \leftarrow G \cup \{\sigma \xrightarrow{g} (c_j, u)\}$ for the minimum-description-length σ
      end if
    end for
    $\Sigma \leftarrow \Sigma \cup \{(\text{var}(c_j), \pi, v) : (\pi, v) \in \textsc{Flatten}(o_j)\}$ ▷ a result witnesses only *later* slots
  end for
  for all pairs (e_s, e_t) sharing a resource where either writes it **do**
    $G \leftarrow G \cup \{e_s \prec e_t\}$ ▷ order edge: no reordering is ever proposed across a conflict
  end for
end for
return G

A Proof of Proposition 1

Fix one threshold $\eta \in \Lambda$ and write $\gamma = \delta/|\Lambda|$. Conditional on $n_\eta = m > 0$ admitted calibration groups, the number of violating groups k_η is binomial with success probability equal to the group-level selective risk r_η at that threshold, because the candidate program, guard, verifier, score, violation rule, and grid were frozen before any calibration group was observed and the original calibration groups are i.i.d. Inverting the one-sided exact binomial test (Clopper & Pearson, 1934) gives the upper bound $U_\eta = \text{Beta}^{-1}(1 - \gamma;\ k_\eta + 1,\ m - k_\eta)$ of eq. (7), which satisfies $\Pr\{r_\eta > U_\eta \mid n_\eta = m\} \le \gamma$. When $m = 0$ the event is impossible under the convention that empty-admission thresholds receive the vacuous bound $U_\eta = 1$. Averaging over the random value of n_η therefore yields $\Pr\{r_\eta > U_\eta\} \le \gamma$ for each fixed threshold. Applying the union bound over the finite grid gives $\Pr\{\exists \eta \in \Lambda : r_\eta > U_\eta\} \le |\Lambda|\gamma = \delta$, i.e. simultaneous coverage with probability at least $1 - \delta$. On that event, any threshold whose upper bound is at most α also satisfies the target selective risk of eq. (4), including the coverage-maximizing threshold returned by Alg. 2. ■

Two scope conditions are worth restating. First, the statement is *per fixed candidate*: Alg. 1 searches over families and returns the first admissible one, and that outer search is not multiplicity-corrected. Second, i.i.d. admitted groups is an assumption about the calibration cohort, not a property the compiler verifies; the grouped split in Alg. 1 makes it plausible by construction but does not establish it under temporal drift.

B Additional Algorithms

Algorithm 3 constructs the typed provenance graph that decides eq. (1). Its two block conditions are asymmetric on purpose: a slot with no witness is treated as a genuine model decision and a slot with more than κ witnesses is treated as ambiguous, so the analysis never resolves an argument by picking the cheapest available explanation.

Algorithm 4 is the runtime counterpart of eq. (3). Cheap deterministic checks reject first and the calibrated gate runs only on what survives them, so the common case costs a registry lookup and

Algorithm 4 DISPATCH — boundary-time admission. Cheap deterministic checks reject first and the calibrated gate runs only on what survives them. Exactly one exit path compacts; the rest return the unmodified agent, and only a dirty post-commit failure raises an incident.

Input: entry state z; runtime context M'; registry $\mathcal{R}$; mode $m \in \{\mathsf{off}, \mathsf{shadow}, \mathsf{live}\}$
Output: observations for the region, or FALLBACK to the baseline agent B

if $m = \mathsf{off}$ **then**
  return FALLBACK ▷ conformance: model-visible input must be byte-identical
end if
$A \leftarrow \mathcal{R}.\text{RESOLVE}(M'.\text{compatibility}, M'.\text{partition})$
if $A = \bot \lor A.\text{lifecycle} \neq \text{ACTIVE} \lor \neg\text{VERIFYSIGNATURE}(A)$ **then**
  return FALLBACK
end if
if $H_A(z, M') \neq 1$ **then**
  return FALLBACK ▷ manifest pins, isolation keys, typed hulls, effect set
end if
if $\text{tools}(A.P) \cap \text{ALREADYOBSERVED} \neq \emptyset$ **then**
  return FALLBACK ▷ the region already ran; its live-ins are not the fitted ones
end if
if $m = \mathsf{live} \land \exists f \in \text{tools}(A.P) : C(f).\textit{quota\_attested} \land \text{SNAPSHOT} = \bot$ **then**
  return FALLBACK ▷ no staging owner, so a discarded attempt is not attestable
end if
if $q_A(z) > \eta_A$ **then**
  return FALLBACK ▷ calibrated abstention on an uncertain input
end if
if $m = \mathsf{shadow}$ **then**
  log would-dispatch; **return** FALLBACK ▷ observe coverage without changing behaviour
end if
$S \leftarrow \text{STAGE.BEGIN}()$; $r \leftarrow \text{INTERPRET}(A.P, z, \text{FACADE}(C))$ ▷ the facade re-checks effects at execution time
if r raised PRECOMMITERROR **then**
  assert $S.\text{REVERSIBLE}()$; **return** FALLBACK
else if r raised POSTCOMMITERROR **then**
  return INCIDENT ▷ an external commitment happened; do not feign rollback
end if
if $V_A(r) \neq 1$ **then**
  discard r; **return** FALLBACK ▷ live-outs, effect multiset, and call count are checked before use
end if
$S.\text{COMMIT}(r.\text{effects})$; **return** r.observations

a guard evaluation. Exactly one exit path compacts; five return the unmodified baseline agent, and one — a post-commit failure — raises an incident rather than feigning a rollback. "Baseline" is byte-exact only where a staging owner holds the commit boundary: the outer runner can restore byte-identical model input, whereas a model-boundary adapter cannot un-emit a response the host has already committed to conversation history.

C CONFIGURATION

All reported admission runs use risk budget $\alpha = 0.05$, confidence budget $\delta = 0.1$, and a fixed grid of $|\Lambda| = 11$ thresholds, which fixes the zero-violation sample requirement at $n_\eta \geq \log(\delta/|\Lambda|)/\log(1-\alpha) = 91.6$, i.e. 92 admitted groups; at exactly $n_\eta = 92$ and $k_\eta = 0$ the bound is $U_\eta = 1 - (0.1/11)^{1/92} = 0.0498 \leq \alpha$. Tightening the confidence budget to $\delta = 0.05$ would raise the requirement to 106 groups, which no study in this paper reaches, so $\delta = 0.1$ is the operative setting throughout and is reported as such rather than rounded to match α. Synthesis uses a 23-operator closed library at depth ≤ 2; region mining uses $w_{\max}$ tool calls per window and requires support across independent scenario groups. The implementation also supports a minimum-distinct-days requirement, which the single-snapshot live study does not exercise (`min_days = 1`). Two terms of eq. (5) are coarser in the implementation than the notation suggests: ρ_{eff} is approximated by the fraction of tool names containing a namespace separator rather than read from the effect catalog, and $|F|$ is substituted by the argument-slot count of the first observed window because ranking precedes

synthesis. Both appear only in a ranking heuristic — no admission decision depends on either, and the effect catalog is consulted where it is load-bearing, in qualification and in the runtime facade.

D Paired Statistics for the Primary Families

Per-record paired differences (compiled minus baseline) on the two families that admit a deep prefix, with 10,000-sample bootstrap intervals and two-sided Wilcoxon signed-rank tests over $n = 30$ paired held-out records:

Family	Endpoint	Paired difference (95% CI)	p
PR-outcome	Total tokens	-2196.7 [-2211.1, -2182.5]	1.7×10^{-6}
	Wall latency	-3889 ms [-4499, -3346]	1.9×10^{-9}
	Estimated cost	$-\$5.11 \times 10^{-4}$	1.7×10^{-6}
Backlog	Total tokens	-2315.0 [-2366.0, -2224.1]	1.7×10^{-6}
	Wall latency	-4428 ms [-5400, -3558]	1.9×10^{-9}
	Estimated cost	$-\$5.44 \times 10^{-4}$	1.7×10^{-6}

Provider-request differences are deterministic by construction (every pair removes the same number of turns), so no rank test is reported for them: the statistic would measure the degeneracy rather than the effect.

E Additional Experimental Detail

Primary workflow families. The three-family result is the main paper's strongest live evidence because it combines distinct tool vocabularies, exact task graders, balanced held-out cohorts, and provider-backed execution. The issue-type family intentionally retains a shallower two-read prefix; the newer pull-request and backlog families admit deeper pre-model compaction. This difference explains the lower resource savings on the first family and is part of the evidence boundary, not noise to average away.

Cross-repository extension. The cross-repository study is narrower than the primary workflow families. Its task uses two read-only tools and a simpler exact outcome contract, which is precisely why a fixed template can remain tied or even more efficient than the learned artifact. We keep the result in the main text because it addresses a reviewer-critical scope question—whether guarded specialization survives beyond one repository—while being explicit about the reduced task complexity.

The position invariant, empirically. An early pilot dispatched a compiled *suffix* region at the entry boundary, violating eq. (2). Because the runtime resolves artifacts before the agent has made the calls the region assumed had already happened, the pilot reordered and duplicated tool calls. The failure is a semantic mismatch between compilation-time position and resolution-time position, not a tuning error, which is why eq. (2) is a hard constraint in the deployed compiler rather than a scoring penalty.

Omitted from the main claim. The repository retains additional studies that are useful scientifically but less suitable for the main narrative:

- a natural-order issue study that exposed a factual continuation miss for a more aggressive three-read artifact,
- exploratory guarded composite synthesis and a follow-up comparison to a fair pre-model manual program,
- a bounded GEPA prompt-optimization study in which GEPA retained its seed, so the combined arm is a replication rather than evidence of interaction, and
- a portfolio pilot and controlled demonstration suite for runtime boundary coverage, including workloads where removing turns *increases* cost through prefix-cache fragmentation.

These studies remain important to the broader artifact, but they either use a weaker risk level, address a narrower engineering question, or are explicitly exploratory.

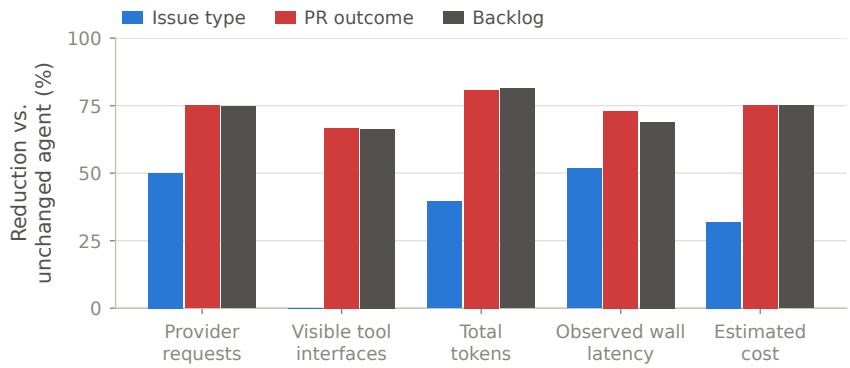

Figure 3: Per-family resource reductions behind table 2, plotted because the spread is the result: admissible prefix depth — not the optimizer — sets the savings, and the two deeper families sit at the feasibility ceiling of eq. (6).

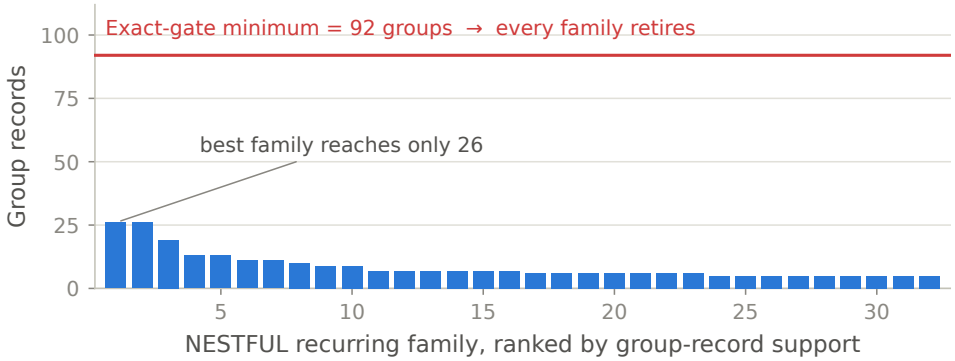

Figure 4: NESTFUL family supports against the 92-group requirement implied by eq. (7) at $\alpha = 0.05$, $\delta = 0.1$, $|\Lambda| = 11$. The largest family reaches 26 groups, so every family retires at calibration even though provenance, synthesis, and replay all succeeded.

F Extended Limitations

The current empirical gate does not yet demonstrate a useful risk–coverage frontier at the registered 5% level. The strongest current runs are mostly all-or-none support tests (fig. 4), and the compiler-wide candidate-search problem is not yet multiplicity-corrected. Likewise, exact task contracts do not certify full semantic equivalence of the final answer, especially once a downstream model continuation is involved. Two further operational caveats belong in the record: the headline live artifact was compiled without a sandbox available to the perturbation suite, so it records `perturbations_claimed: false` rather than a completed challenge, and registry signature verification was configured off in the reported run. These are not hidden defects in the repository; they are the reasons the paper avoids stronger deployment claims.

G Anonymous Artifact Release Plan

Before anonymous submission, package the code and retained artifacts as an anonymous supplementary archive preserving hashes, manifests, scripts, and raw results for the studies cited in the main paper. Direct links to the identified public repository should remain withheld until the review process permits deanonymization.